

BCA152 FreeRTOS Multisensor

Laboratory Report

BCA152 - Microcontrollers
Mindanao State University - Iligan Institute of Technology
College of Computer Studies, Department of Computer Applications

Prepared by: Reuyan, Alwyn Jade

Date: October 5, 2026

1. Introduction and Objectives

This report documents the design, implementation, and verification of an ESP32-based room monitoring system built on FreeRTOS. The system reads temperature, humidity, and ambient light, displays the readings on an OLED screen that a user can navigate with a rotary encoder, raises an audible alarm when temperature leaves a safe range, and tracks room occupancy with a PIR motion sensor to automatically blank the display when the room is empty.

The purpose of the project was not only to produce a working sensor system, but to demonstrate correct use of core FreeRTOS mechanisms: independent tasks with justified priorities, queues for inter-task communication, an event group for state signaling, a mutex for protecting a shared resource, periodic execution using `vTaskDelayUntil`, and a small state machine governing system activity. The project also required separating hardware-independent decision logic from hardware drivers so that core logic could be verified with automated unit tests, and required static analysis and a documented, incremental Git history.

2. System Overview

The system is built around five FreeRTOS tasks that never communicate directly with one another. All data and state changes pass through queues or a shared event group, and the one shared resource used by more than one task (the serial output) is protected by a mutex.

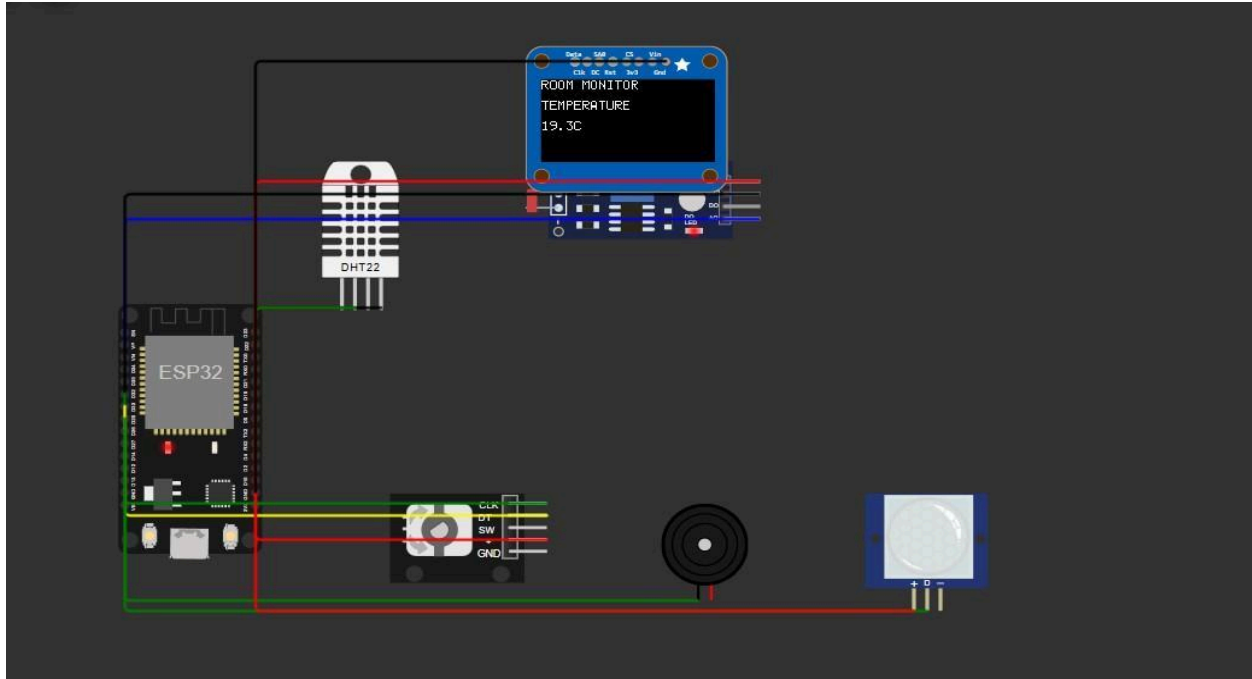
- SensorTask reads the DHT22 (temperature, humidity) and the LDR (light level) every 2 seconds and distributes the reading to two separate queues.
- DisplayTask owns the OLED exclusively and renders whichever measurement mode is currently selected.
- InputTask reads the rotary encoder through an interrupt handler and updates the selected display mode.
- AlarmTask evaluates temperature against configured limits and drives a buzzer when the reading is out of range.
- MotionTask polls the PIR sensor and manages an ACTIVE/INACTIVE system state based on recent motion.

3. Hardware and Circuit Description

Components used, all simulated in Wokwi:

- ESP32 Dev Module
- DHT22 temperature and humidity sensor - GPIO4
- LDR (photoresistor) for ambient light - GPIO36 (VP, ADC1 input-only pin)
- SSD1306 128x64 OLED display over I2C - SDA on GPIO18, SCL on GPIO19
- KY-040 rotary encoder - CLK on GPIO32, DT on GPIO33
- Passive buzzer, driven with LEDC PWM - GPIO25
- PIR motion sensor - GPIO26

The rotary encoder was placed on GPIO32/33 rather than a pin pair closer to the OLED after an earlier wiring attempt put the OLED and the encoder in conflict on the I2C bus; separating them onto distinct, unrelated GPIOs resolved the conflict. The LDR uses GPIO36 specifically because it is one of the ESP32's ADC1 input-only pins, the correct choice for reading an analog sensor. The buzzer is driven through LEDC PWM rather than a simple digital GPIO write because a passive buzzer requires an oscillating signal to produce sound; a plain `gpio_set_level` call did not produce audible output in the simulator.



4. Software Architecture

4.1 Tasks

Each task is implemented in its own source file with hardware-facing driver code kept separate from pure decision logic where applicable (see Section 9).

4.2 Inter-task Communication

- `sensorQueue` (length 5): SensorTask to DisplayTask, carrying the full `SensorData` struct.
- `alarmQueue` (length 5): SensorTask to AlarmTask, the same struct, since a FreeRTOS queue delivers each item to only one receiver and both DisplayTask and AlarmTask need every reading.
- `modeQueue` (length 1, written with `xQueueOverwrite`): InputTask to DisplayTask, holding only the most recently selected display mode.
- `systemEvents` (event group): three bits, `EVENT_ACTIVE`, `EVENT_MOTION`, and `EVENT_ALARM`, set and cleared by MotionTask and AlarmTask and read non-destructively by consuming tasks.
- `serialMutex`: protects `printf/stdout`, since DisplayTask, AlarmTask, and MotionTask can all produce log output and a write is not inherently safe from more than one task at a time.

5. FreeRTOS Task Table

The following table summarizes each task's configuration and the reasoning behind its assigned priority.

Task	Priority	Stack (words)	Blocking Mechanism	Period / Trigger	Priority Justification
AlarmTask	3 (highest)	2048	xQueueReceive(alarmQueue, portMAX_DELAY)	Event-driven, wakes on every new sensor reading	Temperature safety response should not wait behind routine display work. Does negligible work per wake (one comparison, one GPIO/PWM write), so the cost of top priority is minimal.
SensorTask	2	4096	vTaskDelayUntil (periodic)	Every 2000 ms	Needs to run promptly and consistently to keep sampling on schedule, but does not need to preempt the alarm response.
InputTask	2	2048	xQueueReceive(encoderQueue, portMAX_DELAY)	ISR-driven, wakes on encoder rotation	User input should feel responsive, but an idle InputTask spends almost all its time blocked and consumes no CPU while waiting.
MotionTask	2	2048	vTaskDelay (periodic poll)	Every 500 ms	Needs to detect occupancy changes promptly enough that the 15-second timeout feels accurate, without needing to preempt the alarm.
DisplayTask	1 (lowest)	4096	xQueueReceive(sensorQueue, 200 ms timeout)	Every 200 ms (polls modeQueue and systemEvents between sensor waits)	The one task users can tolerate the most latency on. It is intentionally lowest priority so it never blocks anything more time-sensitive.

6. Requirements Traceability Matrix

Temperature alarm limits used throughout: LOW = 18°C, HIGH = 30°C.

FR	Requirement	Implementation	Verification
FR-01	Temperature measurement	SensorTask calls dht22_read() and pushes the result into sensorQueue and alarmQueue.	FT-02 (DHT22 acquisition)
FR-02	Humidity measurement	SensorTask calls dht22_read() and pushes the result into sensorQueue and alarmQueue.	FT-02 (DHT22 acquisition)
FR-03	Ambient-light measurement	SensorTask calls adc_oneshot_read() on the LDR channel and pushes the result into sensorQueue.	FT-03 (LDR acquisition)
FR-04	Motion detection	MotionTask calls gpio_get_level() on the PIR pin and sets EVENT_MOTION in systemEvents.	FT-16 (MOTION page tracks PIR state)
FR-05	OLED display	DisplayTask calls display_draw_string() to render the currently selected measurement on the SSD1306.	FT-05 through FT-08 (OLED page checks)

FR	Requirement	Implementation	Verification
FR-06	Rotary encoder navigation	InputTask maps encoder events through nextDisplayMode()/previousDisplayMode() and writes the result to modeQueue.	FT-06 through FT-08 (encoder navigation) and the display_logic unit tests
FR-07	Temperature alarm	AlarmTask calls evaluateTemperature() and drives the buzzer through LEDC PWM when the result is not NORMAL.	FT-09, FT-10, FT-11 (alarm threshold tests) and the temperature_logic unit tests
FR-08	Activity state	MotionTask sets or clears EVENT_ACTIVE in systemEvents based on evaluateSystemState().	FT-13 (inactivity timeout) and the motion_logic unit tests
FR-09	Automatic inactivity	evaluateSystemState() returns INACTIVE once idle time reaches the 15-second timeout; MotionTask clears EVENT_ACTIVE accordingly.	FT-13 (inactivity timeout) and the motion_logic unit tests
FR-10	Automatic reactivation	A PIR motion reading causes evaluateSystemState() to return ACTIVE immediately; MotionTask sets EVENT_ACTIVE.	FT-14 (reactivation)

7. Functional Verification

All ten functional requirements were exercised directly in the Wokwi simulator across a 20-item test set covering boot behavior, sensor acquisition, queue delivery, OLED rendering, encoder navigation, alarm thresholds, motion-driven activity state, and concurrent serial output. The complete record, including the exact log lines and observations for each test, is maintained separately in docs/functional-verification.md and is summarized rather than reproduced in full here.

Representative results:

- Boot sequence printed the expected startup banner before any sensor output.
- AlarmTask correctly logged HIGH_TEMPERATURE at 31.0°C, LOW_TEMPERATURE at 5.5°C, and NORMAL once the reading returned to 22.5°C, with the OLED header and buzzer state matching in each case.
- MotionTask correctly transitioned to INACTIVE after roughly 16 seconds of no PIR activity, blanking the OLED, and DisplayTask's sensor logging continued uninterrupted during that period.
- The encoder correctly cycled forward through all four display modes and wrapped in both directions.

8. Deliberate Fault Experiments

Three FreeRTOS faults were deliberately introduced, observed, and then reverted, to confirm that the system's correct behavior is not accidental. Full details and analysis are in docs/fault-experiments.md; a summary follows.

8.1 Removing the blocking delay

Commenting out SensorTask's vTaskDelayUntil call caused it to run unblocked as fast as the CPU allowed. Because SensorTask (priority 2) then never yielded, DisplayTask (priority 1) was starved of CPU

time, the terminal was flooded with output, and the OLED visibly flickered. This is a direct demonstration of priority-based CPU starvation on a single-core scheduler.

8.2 Raising a task's priority unnecessarily

InputTask was temporarily raised from priority 2 to priority 5, above every other task including AlarmTask. No degradation was observed, because InputTask spends almost all of its time blocked on an empty queue and only executes briefly when the encoder ISR fires. This illustrates that priority misconfiguration is dangerous specifically when the elevated task performs continuous, non-blocking work; an event-driven task remains safe even at an incorrectly high priority.

8.3 Removing the serial mutex

The xSemaphoreTake/xSemaphoreGive pairs protecting printf calls in DisplayTask, AlarmTask, and MotionTask were removed simultaneously, and alarm and motion events were triggered in rapid succession to try to force overlapping writes. No interleaved or garbled output was observed. The most likely explanation is that ESP-IDF's printf path serializes writes at the UART driver level in this simulated, blocking-UART configuration. This does not make the application-level mutex unnecessary: it documents the shared resource explicitly and would matter on hardware configurations where the UART driver does not provide that same implicit serialization.

9. Separation of Logic and Unit Testing

Three decision functions were extracted from their hardware-facing task files into standalone, dependency-free modules so they could be compiled and tested on the host machine without any ESP-IDF or FreeRTOS headers: evaluateTemperature() in temperature_logic.cpp, nextDisplayMode() and previousDisplayMode() in display_logic.cpp, and evaluateSystemState() in motion_logic.cpp.

A native PlatformIO test environment was configured separately from the ESP32 build target. Eighteen unit tests were written across three suites: seven for temperature threshold behavior (including exact boundary values at 18°C and 30°C), six for display mode cycling (forward, reverse, and full wraparound), and five for system-state evaluation (idle timeout behavior and motion override). All eighteen tests pass:

- pio test -e native -> 18 test cases: 18 succeeded

10. Static Code Analysis

Static analysis was performed with cppcheck via pio check -e esp32dev. An initial run failed to complete because cppcheck attempted to parse a GCC toolchain header outside the project's own source and encountered a syntax error unrelated to any project code. Adding check_skip_packages = yes to platformio.ini restricted analysis to the project's own src/ and include/ directories. The resulting run reported No defects found.

11. Engineering Decisions and Justifications

- Native ESP-IDF drivers instead of Arduino libraries: available PlatformIO libraries for the DHT22 and SSD1306 depend on the Arduino framework, which was not permitted, so a bit-bang DHT22

driver and a minimal SSD1306 I2C driver using the ESP-IDF v6.0.1 `i2c_master` API were written directly.

- LEDC PWM for the buzzer: a plain digital GPIO write did not produce audible output for the passive buzzer in the simulator; LEDC PWM at 2 kHz resolved this and is also the correct approach on real hardware.
- Two queues fed from one task: because a FreeRTOS queue delivers each item to only one receiver, `SensorTask` pushes every reading into both `sensorQueue` and `alarmQueue` so that `DisplayTask` and `AlarmTask` can each consume it independently.
- Event group over per-task notifications: a single event group with three named bits (`EVENT_ACTIVE`, `EVENT_MOTION`, `EVENT_ALARM`) allows any task to query system state without needing a direct reference to the task that produces it.
- Polled PIR versus interrupt-driven encoder: the PIR module already outputs a debounced digital level, so polling it every 500 ms is simple and reliable. The encoder produces raw quadrature edges that require an interrupt handler to capture reliably.
- Mutex-protected serial output: `DisplayTask`, `AlarmTask`, and `MotionTask` can all log at effectively unpredictable times relative to one another, so a mutex around `printf` was added to guarantee that log lines are not interleaved, regardless of how the underlying UART driver behaves.
- Delay between consecutive I2C transactions: `display_init()` and `display_clear()` each fire many I2C transactions in immediate succession with no gap. Comparing exact log timestamps against the code showed this burst pattern, not the choice of pins, was the actual cause of a recurring I2C timeout warning. Adding a single-tick `vTaskDelay` between transactions in `display.cpp` resolved it completely, verified by a before/after comparison of identical boot and `INACTIVE`-transition runs.

12. Limitations

- The I2C driver initially printed a recurring warning ("GPIO 18 is not usable, maybe conflict with others") during `display_init()` and any call to `display_clear()`. Tracing the exact timestamps against the code showed the warning only ever appeared during bursts of many back-to-back I2C transactions with no delay between them (the 28-command init sequence, and the 88-transaction full-screen clear), never during the steady 2-second sensor update cycle where no I2C traffic occurs. Adding a 1-tick `vTaskDelay` between each I2C transaction in `display.cpp` eliminated the warning entirely, confirmed by comparing identical boot and `INACTIVE`-transition sequences before and after the change. This ruled out a pin-conflict or wiring explanation and confirmed the cause was Wokwi's simulated I2C peripheral not reliably keeping up with rapid consecutive transactions under the ESP-IDF v6.0.1 `i2c_master` driver.
- The hand-written DHT22 bit-bang driver can occasionally produce a transient invalid reading when the simulated sensor value is changed very rapidly.
- Rotating the encoder does not reset the inactivity timer; only PIR motion counts as activity, per FR-09/FR-10, which is intentional rather than an oversight.
- A failed DHT22 read currently falls back to a temperature of 0°C rather than being excluded from the alarm evaluation, which can incorrectly trigger `LOW_TEMPERATURE`; a more careful implementation would skip the alarm check entirely on an invalid read.

- There is up to approximately 700 ms of latency between an actual motion event and the display's response, due to MotionTask's 500 ms poll interval and DisplayTask's 200 ms event-group check interval.
- Wokwi reports 2 MB of flash while the targeted ESP32 Dev Module has 4 MB; this is a simulator default mismatch with no functional effect.

13. Conclusion

The completed system satisfies all ten functional requirements, uses FreeRTOS tasks, queues, an event group, and a mutex in ways that are individually justified rather than applied by default, and includes automated unit tests and static analysis as part of its verification. Deliberate fault experiments confirmed that the system's correct behavior depends on its actual design choices rather than being coincidental, and the project's Git history reflects genuine incremental development across each stage of the build.

14. References

- ESP-IDF Programming Guide - <https://docs.espressif.com/projects/esp-idf/en/latest/>
- FreeRTOS Documentation - https://www.freertos.org/Documentation/RTOS_book.html
- Wokwi Simulator Docs - <https://docs.wokwi.com/>
- PlatformIO Docs - <https://docs.platformio.org/>
- SSD1306 Datasheet - <https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>
- DHT22 Datasheet - <https://www.sparkfun.com/datasheets/Sensors/Temperature/DHT22.pdf>